PlantillasValidadoras4GL

Manual de Uso del Programa

Validacion de plantillas Excel con Genero 4GL e Informix 14.10

Parte 1: Sin Python (100% Genero 4GL - SpreadsheetML)

Parte 2: Con Python (archivos .xlsx nativos)

Version 2.0 - 12/10/2026

Compatibilidad: Genero BDL 1.x / 2.x / 3.x / 4.x

Guia paso a paso para usar el sistema de validacion de plantillas.

Dos modos: sin Python (100% 4GL) y con Python (.xlsx).

PARTE 1: SIN PYTHON (100% Genero 4GL)

Usa archivos SpreadsheetML 2003 (extension .xls, contenido XML).

Excel los abre con diseno intacto. Genero los lee y escribe sin librerias.

1. Requisitos

Componente	Version	Notas
Informix	14.10	Edicion Developer o superior, idsxm
Genero BDL	1.x / 2.x / 3.x / 4.x	Cualquiera, modo escritorio (GDC)
Excel	2003 o superior	Abre SpreadsheetML
Windows	10/11	Para build.bat y RUN "copy" / RUN "

No se necesita Python ni ninguna libreria extra.

2. Instalacion de la base de datos

```
REM 1. Iniciar Informix (Docker)
docker start ifx
```

```
REM 2. Crear esquema y tablas (en orden)
dbaccess sysmaster db/01_esquema.sql
dbaccess banco db/02_tablas.sql
dbaccess banco db/03_catalogos.sql
dbaccess banco db/04_sp_xml.sql
dbaccess banco db/reglas_validacion.sql
```

Verificacion:

```
-- Debe devolver 1 fila
SELECT COUNT(*) FROM banco_cliente;
SELECT COUNT(*) FROM cat_tipo_documento;
-- Debe devolver XML
SELECT genxmlqueryhdr('t','SELECT 1 AS c FROM systables WHERE tabid=1')
FROM systables WHERE tabid=1;
```

Si el ultimo SELECT devuelve NULL, el VP idsxmlvp no esta activo.

Activarlo segun documentacion de Informix 14.10.

3. Compilacion en Genero

Elegir la carpeta segun tu version instalada:

```
REM Genero 1.x
cd genero\bdl_1x
build.bat

REM Genero 2.10/2.11
cd genero\bdl_2x
build.bat

REM Genero 3.x
cd genero\bdl_3x
build.bat

REM Genero 4.x (modo escritorio)
cd genero\bdl_4x
build.bat
```

Cada build.bat compila estos 6 modulos:

```
validacion.4gl          -> funciones val_*
file_compat.4gl         -> archivos compatible 1.x/2.x/3.x/4.x
spreadsheetml.4gl       -> leer/escribir SpreadsheetML
validador.4gl           -> flujo validar + exportar
test_spreadsheetml.4gl  -> 9 tests
principal.4gl           -> menu principal
```

Si build.bat termina con Compilacion exitosa!, ejecutar:

```
fglrun principal
```

4. Uso paso a paso (flujo normal)

Paso 1: Abrir la plantilla modelo

Abrir en Excel:

```
plantillas\plantilla_minima.xls
```

Contenido:

Fila	Contenido
1	Titulo: BANCO NACION - Clientes
2	Encabezados: Tipo Doc, Nro Doc, Apellidos, Nombres
3	Tags: {{tipo_doc}}, {{num_doc}}, {{apellidos}}, {{no
4+	Datos de ejemplo

Los tags de la fila 3 le dicen al sistema que campo es cada columna.

No borrar la fila de tags.

Paso 2: Llenar datos

Escribir los datos del cliente desde la fila 4 en adelante,

una fila por cliente:

```
Tipo Doc | Nro Doc  | Apellidos | Nombres
DNI      | 30123456 | PEREZ     | JUAN CARLOS
DNI      | 30789012 | GARCIA    | MARIA ELENA
```

Guardar como:

```
plantillas\plantilla_datos.xls
```

(Puede ser cualquier nombre y cualquier carpeta.)

Paso 3: Validar en Genero

```
fglrun principal
  -> 1. Validar Archivo SpreadsheetML
  -> Escribir ruta: plantillas\plantilla_datos.xls
  -> Validar
```

El sistema hace por cada fila:

```
1. Lee el tag de la columna  -> sabe que campo es
2. Extrae el valor de la celda
3. Aplica el validador que corresponde:
     tipo_doc  -> val_catalogo(cat_tipo_documento)
     num_doc   -> val_documento (alfanumerico 5-20)
     apellidos -> val_nombre (solo letras 2-60)
     nombres   -> val_nombre (solo letras 2-60)
```

Paso 4a: Si todo esta OK

Mensaje:

```
========================================
Validacion completa
  Filas OK: 3
  Filas con error: 0
========================================
```

Opciones:

```
-> 3. Exportar Datos Validados
   Origen:  plantillas\plantilla_datos.xls
   Destino: salida\datos_validados.xls
```

El archivo exportado no tiene tags, solo titulo, encabezados y datos.

Abrirlo en Excel para verificar.

Archivo de ejemplo ya generado:

```
salida\datos_validados.xls
```

Paso 4b: Si hay errores

Mensaje por cada fila con error:

```
Fila 4: Tipo documento invalido: INVALIDO
Fila 4: Documento invalido: 123
```

El sistema genera un archivo con 2 hojas:

Hoja	Contenido
Datos	Mismos datos, celdas con error en rojo
Errores	Tabla: Fila, Campo, Valor Ingresado, Motivo del Erro

Archivo de ejemplo:

```
plantillas\plantilla_errores.xls
```

Abrirlo en Excel, ir a la hoja Errores, corregir los datos

en la hoja Datos (o en el archivo original) y volver al Paso 3.

5. Tests (9 escenarios)

```
fglrun principal
  -> 4. Ejecutar Tests
```

#	Test	Que prueba	Resultado esperado
1	Leer archivo	Parsea SpreadsheetML	Filas y celdas correctas
2	Validar OK	plantilla_datos.xls	Todas las filas OK
3	Validar errores	Datos invalidos	Errores detectados
4	Archivo vacio	XML sin filas	Error controlado
5	Sin tags	XML sin {{}}	Error controlado
6	Caracteres especiales	Ñ, tildes, UTF-8	Se leen sin romper
7	Exportar	Genera .xls limpio	Sin tags en salida
8	Validadores	Todos los val_* con OK y E	Cada uno responde bien
9	Inexistente	Ruta que no existe	Error controlado

6. Tabla de validadores

Validador	Regla	Ejemplo OK	Ejemplo ERROR
val_nombre	Solo letras y espacios, 2-	PEREZ, JUAN CARLOS	PEREZ123, JUAN@
val_documento	Alfanumerico, 5-20	30123456	123
val_fecha	DD/MM/YYYY, 1900-hoy	15/03/1985	32/13/2025
val_email	Con @ y dominio	juan@email.com	juan@, juanemail.com
val_telefono	7-20 digitos	011-4567-8901	123
val_anio	Entero 1900-actual	1985	1800
val_monto	Decimal >= 0	15000.50	-100
val_catalogo	Existe en tabla	DNI en cat_tipo_documento	INVALIDO

7. Solucion de problemas (sin Python)

Sintoma	Causa	Solucion
Excel dice "file format and extensi	Extension .xls con contenido XML	Dar Yes (es normal en SpreadsheetML
"Problems During Load / Table"	XML con elementos no soportados (Da	Usar XML limpio como plantilla_mini
Se ve codigo XML en vez de hoja	Se abrio con .xml o en navegador	Renombrar a .xls y abrir en Excel
"No se encontraron tags"	Fila de {{campo}} borrada o con esp	Verificar fila 3: {{tipo_doc}} exac
"Archivo no encontrado"	Ruta con espacios sin comillas	Escribir la ruta completa entre com
fglcomp no reconocido	FGLDIR mal configurado	Editar build.bat con la ruta de tu
SP genxmlqueryhdr devuelve NULL	idsxmlvp inactivo	Activar el VP XML en Informix

PARTE 2: CON PYTHON (archivos .xlsx nativos)

Usa archivos .xlsx reales (formato Office Open XML) con la libreria openpyxl. Python solo lee/escribe Excel; la validacion sigue siendo 4GL (val_*).

1. Requisitos extra

Componente	Version	Instalacion
Python	3.8 o superior	https://www.python.org/downloads/
openpyxl	ultima	pip install openpyxl

Todo lo de la Parte 1 (Informix, Genero, Excel) sigue igual.

2. Scripts necesarios (carpeta tools/)

Crear estos 3 scripts (o recuperarlos del historial del repo):

2.1 tools/leer_xlsx.py - Excel -> datos para 4GL

```
#!/usr/bin/env python3
"""Lee un .xlsx y escribe Filas Validadas en CSV para que 4GL lo lea.
Uso: python leer_xlsx.py <archivo.xlsx> <hoja> <fila_tags> <salida.csv>
"""
import sys, csv
from openpyxl import load_workbook

archivo, hoja, fila_tags, salida = sys.argv[1], sys.argv[2], int(sys.argv[3]), sys.argv[4]

wb = load_workbook(archivo, data_only=True)
ws = wb[hoja] if hoja in wb.sheetnames else wb.active

# 1. Leer tags de la fila indicada
tags = []
for cell in ws[fila_tags]:
    v = str(cell.value or "").strip()
    tags.append(v[2:-2] if v.startswith("{{") and v.endswith("}}") else "")

# 2. Escribir datos (filas siguientes) como CSV: fila;tag;valor
with open(salida, "w", encoding="utf-8", newline="") as f:
    w = csv.writer(f, delimiter=";")
```

```
    w.writerow(["fila", "tag", "valor"])
    for r in range(fila_tags + 1, ws.max_row + 1):
        for c, tag in enumerate(tags, 1):
            if not tag:
                continue
            v = ws.cell(row=r, column=c).value
            w.writerow([r, tag, "" if v is None else str(v).strip()])
print(f"OK: {salida}")
```

2.2 tools/escribir_xlsx.py - datos validados -> .xlsx limpio

```
#!/usr/bin/env python3
"""Genera un .xlsx limpio (sin tags) desde un CSV de datos validados.
Uso: python escribir_xlsx.py <datos.csv> <salida.xlsx> <titulo>
"""
import sys, csv
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill

datos_csv, salida, titulo = sys.argv[1], sys.argv[2], sys.argv[3]

# Agrupar por fila
filas = {}
orden_tags = []
with open(datos_csv, encoding="utf-8") as f:
    for row in csv.DictReader(f, delimiter=";"):
        filas.setdefault(int(row["fila"]), {})[row["tag"]] = row["valor"]
        if row["tag"] not in orden_tags:
            orden_tags.append(row["tag"])

wb = Workbook()
ws = wb.active
ws.title = "Datos"
hdr_font = Font(bold=True, color="FFFFFF")
hdr_fill = PatternFill(start_color="003366", end_color="003366", fill_type="solid")
ws.append([titulo])
for c, t in enumerate(orden_tags, 1):
    cell = ws.cell(row=2, column=c, value=t)
    cell.font, cell.fill = hdr_font, hdr_fill
for vals in filas.values():
    ws.append([vals.get(t, "") for t in orden_tags])
wb.save(salida)
print(f"OK: {salida}")
```

2.3 tools/escribir_errores_xlsx.py - archivo con hoja Errores

```
#!/usr/bin/env python3
"""Copia el .xlsx original y agrega hoja Errores con el detalle.
Uso: python escribir_errores_xlsx.py <original.xlsx> <errores.csv> <salida.xlsx>
errores.csv columnas: fila;campo;valor;motivo;validador
"""
import sys, csv, shutil
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill

original, errores_csv, salida = sys.argv[1], sys.argv[2], sys.argv[3]
shutil.copy2(original, salida)
```

```
wb = load_workbook(salida)
ws = wb.create_sheet("Errores")
hdr_font = Font(bold=True, color="FFFFFF")
hdr_fill = PatternFill(start_color="CC0000", end_color="CC0000", fill_type="solid")
for c, h in enumerate(["Fila", "Campo", "Valor", "Motivo", "Validador"], 1):
    cell = ws.cell(row=1, column=c, value=h)
    cell.font, cell.fill = hdr_font, hdr_fill
r = 2
with open(errores_csv, encoding="utf-8") as f:
    for row in csv.DictReader(f, delimiter=";"):
        ws.cell(row=r, column=1, value=row["fila"])
        ws.cell(row=r, column=2, value=row["campo"])
        ws.cell(row=r, column=3, value=row["valor"])
        ws.cell(row=r, column=4, value=row["motivo"])
        ws.cell(row=r, column=5, value=row["validador"])
        r += 1
wb.save(salida)
print(f"OK: {salida}")
```

3. Flujo con Python paso a paso

Paso 1: Preparar la plantilla .xlsx

- Abrir Excel, crear la planilla del banco con su diseno libre.
- En una fila (ej. fila 3) escribir los tags: {{tipo_doc}},

{{num_doc}}, {{apellidos}}, {{nombres}}.

- Guardar como plantilla_banco.xlsx.

Paso 2: Llenar datos

El usuario llena las filas siguientes (fila 4 en adelante) y guarda

como datos_banco.xlsx.

Paso 3: Extraer datos con Python

```
python tools\leer_xlsx.py datos_banco.xlsx Datos 3 datos.csv
```

Genera datos.csv:

```
fila;tag;valor
4;tipo_doc;DNI
4;num_doc;30123456
4;apellidos;PEREZ
4;nombres;JUAN CARLOS
...
```

Paso 4: Validar en Genero

En 4GL, leer datos.csv linea por linea (con OPEN ... FOR INPUT

como en file_compat.4gl) y aplicar el val_* que corresponde

a cada tag:

```
tag = tipo_doc  -> val_catalogo("cat_tipo_documento","codigo",valor)
tag = num_doc   -> val_documento(valor)
tag = apellidos -> val_nombre(valor)
tag = nombres   -> val_nombre(valor)
```

Por cada celda escribir una linea en resultado.csv:

```
fila;campo;valor;estado;motivo
4;tipo_doc;DNI;OK;
4;num_doc;123;ERROR;Documento debe tener 5-20 caracteres
```

Paso 5a: Si todo OK -> exportar limpio

```
python tools\escribir_xlsx.py datos.csv salida\datos_validados.xlsx "Datos Validados"
```

Abrir salida\datos_validados.xlsx en Excel: titulo, encabezados

y datos, sin tags.

Paso 5b: Si hay errores -> archivo con hoja Errores

```
python tools\escribir_errores_xlsx.py datos_banco.xlsx resultado.csv salida\datos_errores.xlsx
```

Abrir salida\datos_errores.xlsx: hoja Datos (original) +

hoja Errores (Fila, Campo, Valor, Motivo, Validador).

Corregir y volver al Paso 3.

4. Comparacion: sin Python vs con Python

Aspecto	Sin Python (Parte 1)	Con Python (Parte 2)
Formato	SpreadsheetML .xls	Office Open XML .xlsx
Aviso de Excel al abrir	Si ("format and extension don't mat	No
Validaciones Excel nativas	No (se validan en 4GL)	Si (listas, fechas con openpyxl)
Lectura en 4GL	Directa (spreadsheetml.4gl)	Via CSV puente
Escritura en 4GL	Directa (spm_generar_xml)	Via scripts Python
Dependencias	Ninguna	Python + openpyxl
Compatibilidad Genero	1.x/2.x/3.x/4.x	1.x/2.x/3.x/4.x
Recomendado para	Entornos sin Python, SICU existente	Usuarios que exigen .xlsx real

5. Solucion de problemas (con Python)

Sintoma	Causa	Solucion
No module named openpyxl	Libreria no instalada	pip install openpyxl
python no se reconoce	Python no en PATH	Reinstalar marcando "Add to PATH"
CSV con caracteres raros	Encoding	Abrir/guardar siempre en UTF-8
Tags no detectados	Espacios: {{ tipo_doc }}	Deben ser exactos: {{tipo_doc}}
Hoja no encontrada	Nombre distinto	Pasar el nombre exacto de la hoja

Chequeo rapido (ambos modos)

- [ ] BD creada y catalogos cargados
- [ ] build.bat compila sin errores en tu version Genero
- [ ] Template abre en Excel (con aviso Yes en modo sin Python)
- [ ] Fila de tags intacta
- [ ] Validacion OK detecta datos buenos
- [ ] Validacion ERROR detecta datos malos y explica el motivo
- [ ] Exportacion genera archivo limpio sin tags